Manual Testing Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Software Tester with 4+ years of experience

Questions in your original document	**350**
Exact duplicates removed (same question, different wording of punctuation or case)	**6**
Near-duplicates merged (same question asked in different words)	**110**
Unique questions in this PDF	**234**

Section 1: Critical (Must Prepare)	38 questions
Section 2: High Priority	74 questions
Section 3: Medium Priority	92 questions
Section 4: Low Priority / Quick Revision	30 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 38 questions	Asked in almost every interview at 4+ years. A weak answer here usually decides the outcome, and interviewers follow up with examples from your own project. Prepare a crisp definition plus a real-life example for each.
High Priority 74 questions	Asked frequently, mostly as process, scenario and 'how would you handle' questions. These separate an experienced tester from a fresher, so answer with a structured approach (steps, risks, trade-offs).
Medium Priority 92 questions	Breadth topics. Asked occasionally, usually as follow-ups to your earlier answers or to probe a specific area on your resume. One clear paragraph per question is enough.
Low Priority / Quick Revision 30 questions	Fresher-level definitions and niche topics. Rarely asked at 4+ years, but worth a one-line answer so you are never caught off guard.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
Defect Management	Do you own the defect process end to end? Expect real stories: rejected bugs, leaks, release-day blockers.
Smoke/Sanity/Regression & Automation	Can you build and optimize suites, and decide what to automate and what to leave manual?
Test Design & Documentation	Can you design coverage systematically (EP/BVA, RTM, plan vs strategy) instead of guessing?
Situational & Behavioral	Judgement under pressure: vague requirements, deadlines, conflict, unstable environments.
Agile & Scrum	Where QA fits in sprints: estimation, late stories, scope change, release decisions.
Types & Levels of Testing	Clean definitions and the 'how is X different from Y' comparisons.
Scenario-Based Feature Testing	A structured approach: functional, negative, boundary, security, performance and UX angles.
Integration, DB, API, Perf & Security	Do you think beyond the UI: data consistency, concurrency, injection, slow back ends?

Note: one line in your original file ('Manual Testing - Designed test cases for CSV validation...') was a resume bullet, not a question. It has been converted into an interview question you can expect from it.

Table of Contents

Section 1: Critical (Must Prepare) (38 questions) . . . 5
Defect Management & Bug Life Cycle (7) . . . 5
Smoke / Sanity / Regression & Automation Decisions (7) . . . 6
Test Design, Planning & Documentation (9) . . . 7
Situational & Behavioral Scenarios (5) . . . 8
Agile & Scrum (5) . . . 8
Types & Levels of Testing (1) . . . 9
Testing Fundamentals, SDLC & STLC (2) . . . 9
Scenario-Based Feature Testing (E-commerce & Real World) (2) . . . 10

Section 2: High Priority (74 questions). . . . 11
Defect Management & Bug Life Cycle (5) . . . 11
Smoke / Sanity / Regression & Automation Decisions (8) . . . 11
Test Design, Planning & Documentation (8) . . . 12
Situational & Behavioral Scenarios (4) . . . 13
Agile & Scrum (10) . . . 14
Types & Levels of Testing (11) . . . 15
Testing Fundamentals, SDLC & STLC (1) . . . 16
Scenario-Based Feature Testing (E-commerce & Real World) (14) . . . 16
Integration, DB, API, Performance & Security Scenarios (5) . . . 18
Metrics, Coverage & Reporting (5) . . . 19
JIRA & Defect Tracking Tools (3). . . . 19

Section 3: Medium Priority (92 questions) . . . 20
Defect Management & Bug Life Cycle (6) . . . 20
Smoke / Sanity / Regression & Automation Decisions (3) . . . 20
Test Design, Planning & Documentation (4) . . . 21
Situational & Behavioral Scenarios (1) . . . 21
Agile & Scrum (19) . . . 21
Types & Levels of Testing (12) . . . 23
Testing Fundamentals, SDLC & STLC (7) . . . 24
Scenario-Based Feature Testing (E-commerce & Real World) (17) . . . 25
Integration, DB, API, Performance & Security Scenarios (5) . . . 27
Metrics, Coverage & Reporting (4) . . . 27
JIRA & Defect Tracking Tools (6) . . . 28
QA Mindset, Leadership & Career (8) . . . 28

Section 4: Low Priority / Quick Revision (30 questions) . . . 30
Defect Management & Bug Life Cycle (5) . . . 30
Smoke / Sanity / Regression & Automation Decisions (2) . . . 30
Test Design, Planning & Documentation (1) . . . 30
Types & Levels of Testing (8) . . . 31
Testing Fundamentals, SDLC & STLC (1) . . . 31
Scenario-Based Feature Testing (E-commerce & Real World) (2) . . . 31
Integration, DB, API, Performance & Security Scenarios (1) . . . 32
Metrics, Coverage & Reporting (5) . . . 32
QA Mindset, Leadership & Career (5) . . . 32

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
Defect Management & Bug Life Cycle	7 Qs, p.5	5 Qs, p.11	6 Qs, p.20	5 Qs, p.30	**23**
Smoke / Sanity / Regression & Automation Decisions	7 Qs, p.6	8 Qs, p.11	3 Qs, p.20	2 Qs, p.30	**20**
Test Design, Planning & Documentation	9 Qs, p.7	8 Qs, p.12	4 Qs, p.21	1 Qs, p.30	**22**
Situational & Behavioral Scenarios	5 Qs, p.8	4 Qs, p.13	1 Qs, p.21	-	**10**
Agile & Scrum	5 Qs, p.8	10 Qs, p.14	19 Qs, p.21	-	**34**
Types & Levels of Testing	1 Qs, p.9	11 Qs, p.15	12 Qs, p.23	8 Qs, p.31	**32**
Testing Fundamentals, SDLC & STLC	2 Qs, p.9	1 Qs, p.16	7 Qs, p.24	1 Qs, p.31	**11**
Scenario-Based Feature Testing (E-commerce & Real World)	2 Qs, p.10	14 Qs, p.16	17 Qs, p.25	2 Qs, p.31	**35**
Integration, DB, API, Performance & Security Scenarios	-	5 Qs, p.18	5 Qs, p.27	1 Qs, p.32	**11**
Metrics, Coverage & Reporting	-	5 Qs, p.19	4 Qs, p.27	5 Qs, p.32	**14**
JIRA & Defect Tracking Tools	-	3 Qs, p.19	6 Qs, p.28	-	**9**
QA Mindset, Leadership & Career	-	-	8 Qs, p.28	5 Qs, p.32	**13**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (38 questions)

Asked in almost every interview at 4+ years. A weak answer here usually decides the outcome, and interviewers follow up with examples from your own project. Prepare a crisp definition plus a real-life example for each.

Defect Management & Bug Life Cycle (7)

1. Explain the Defect (Bug) Life Cycle and its states. **[asked x8 in your bank]**

   > **Answer:** The journey of a defect from discovery to closure: New > Assigned > Open > Fixed > Retest > Verified > Closed. Side paths: Reopened (fix failed), Rejected/Not a Bug, Duplicate, Deferred, Cannot Reproduce.
   >
   > **Example:** Coupon is not applied at checkout: logged as New, dev fixes it, I retest on the new build and close it. If the discount still fails, I reopen it with fresh evidence.

2. What is the difference between Severity and Priority? Give examples (high severity/low priority and low severity/high priority). **[asked x9 in your bank]**

   > **Answer:** Severity is the technical impact of the defect; Priority is how soon the business wants it fixed. Severity is set by QA, priority by the Product Owner/business.
   >
   > **Example:** High severity / low priority: the app crashes only on an obsolete browser. Low severity / high priority: a spelling mistake in the brand name on the home page before a campaign launch.

3. How do you write a good bug report? What are the most important things to define in a bug? **[asked x3 in your bank]**

   > **Answer:** A good bug is clear, reproducible and complete: precise title, environment/build, numbered steps, expected vs actual result, severity/priority, and evidence (screenshot, video, logs, API response).
   >
   > **Example:** Title: 'Order total shows Rs.1,200 instead of Rs.1,080 after applying SAVE10 on cart'. Then steps, build number, browser and a screenshot.

4. What do you do when a developer rejects or disagrees with your defect ("Not a bug", "Rejected")? How do you convince them? **[asked x6 in your bank]**

   > **Answer:** I stay factual: recheck the requirement/AC, reproduce it again, and share evidence (steps, video, logs). If the doubt remains, I discuss it in a quick call or triage with the BA/PO to settle the expected behaviour. If it is truly not a bug, I accept it and note the learning.
   >
   > **Example:** Dev says 'working as designed' for free shipping above Rs.499. I show the BRD line saying Rs.399, and the PO confirms it, so the bug stays valid.

5. A production issue is reported that you did not catch in testing (or you missed a scenario). How do you handle it? **[asked x4 in your bank]**

   > **Answer:** First reproduce and assess the impact, then support the hotfix with a quick RCA. I add the missed scenario to the regression suite and fix the process gap (review, test data or coverage). Blame is not the goal; prevention is.
   >
   > **Example:** A production defect lets users place an order with an expired card. I reproduce it, verify the hotfix, add 'expired card' cases to the payment suite and share the RCA.

6. How do you handle a situation where a high-severity bug is found just before release?

   > **Answer:** Report it immediately with impact and evidence, and hold a quick triage with Dev, PO and the release manager. The decision (fix now, workaround, or delay the release) is a business call; QA gives the risk and the options.
   >
   > **Example:** Payment gets debited but the order is not created, found a day before release. I raise it as a blocker, the team patches it and I rerun the payment regression before we go live.

7. What is the most challenging bug you have found and how did you debug it? **[asked x2 in your bank]**

> **Answer:** Pick a real case: an intermittent or data-dependent defect. Explain how you isolated it: narrowed the steps, checked logs, DB and network calls, and compared environments, then handed the developer a precise reproduction.
>
> **Example:** The cart total was occasionally wrong. I found it happened only when a quantity was updated while a discount was being applied (a race condition), proved it with API logs and the dev fixed the sequence.

Smoke / Sanity / Regression & Automation Decisions (7)

8. What is the difference between Smoke, Sanity and Regression testing? **[asked x6 in your bank]**

> **Answer:** Smoke: a quick check that a new build is stable enough to test (critical paths). Sanity: a narrow check that a specific fix or change works. Regression: a wide check that existing features still work after changes.
>
> **Example:** Smoke: login, search, add to cart, checkout opens. Sanity: only the fixed coupon bug. Regression: the entire cart, payment and order modules.

9. What is the difference between Regression Testing and Retesting? **[asked x3 in your bank]**

> **Answer:** Retesting re-executes the failed test to confirm that a specific defect is fixed. Regression testing checks that the fix or change did not break other existing functionality.
>
> **Example:** Retest: the failed 'Apply promo code' case after the fix. Regression: cart, tax and payment tests after that fix.

10. You have 250 manual test cases. On what basis will you segregate them into Smoke, Sanity and Regression suites, and when is each executed? **[asked x2 in your bank]**

> **Answer:** Smoke: about 10-15% (critical end-to-end flows), run on every new build. Sanity: cases related to the changed area, run after a fix or minor change. Regression: the full set, run before release or after major changes. I choose by business criticality, defect history and usage.
>
> **Example:** Smoke: login > search > cart > pay. Sanity: the discount module after a coupon fix. Regression: all 250 cases before release.

11. How do you decide the regression suite scope?

> **Answer:** I base it on the change: impacted modules (impact analysis), high-risk and business-critical flows, areas with a bug history, and integration points. Then I trim low-value and duplicate cases to keep it manageable.
>
> **Example:** A change in the tax calculation means regression on cart total, checkout, invoice, refund and order history.

12. How do you decide whether a test case is a good candidate for automation? (e.g. you have 1000 test cases, which do you automate? Long-running scenario: manual or automation?) **[asked x3 in your bank]**

> **Answer:** Automate stable, repetitive, high-risk, data-driven cases that run on every release. Keep manual for exploratory, usability, one-time and rapidly changing cases. Long-running but stable scenarios are good for automation (run overnight in CI).
>
> **Example:** Automate login, search and checkout regression. Keep the new homepage banner design and a first-time UX review manual.

13. Your regression suite takes 1.5 to 5 hours but stakeholders need results in under 1 hour. How do you reduce execution time? **[asked x2 in your bank]**

> **Answer:** Run tests in parallel (grid or cloud), remove duplicate and obsolete cases, split into risk-based tiers (a critical subset in under 1 hour, the rest nightly), use API/data setup instead of UI steps, and replace hard waits with smart waits.
>
> **Example:** Run only the top 30% critical cart-checkout-payment tests on every commit, and the full suite overnight.

14. How do you handle flaky automation tests? **[asked x2 in your bank]**

> **Answer:** I find the root cause first: timing and sync issues, unstable locators, shared test data or environment problems. Fix with explicit waits, stable locators and isolated data. Quarantine unstable tests until fixed instead of ignoring failures.
>
> **Example:** An 'Add to cart' test fails sometimes because the button loads late. I replace the fixed sleep with a wait for the button to be clickable.

Test Design, Planning & Documentation (9)

15. What is a Test Plan? What does it include and what are the steps to create one? **[asked x4 in your bank]**

> **Answer:** A Test Plan is a document defining scope, objectives, approach, resources, schedule, environment, entry/exit criteria, risks and deliverables. Steps: analyse requirements, define scope and strategy, estimate effort, plan resources and schedule, identify risks, then review and get sign-off.
>
> **Example:** For the new 'Wishlist' feature: scope (add/remove/move to cart), Chrome and Android, 2 testers for 5 days, with the risk of a delayed build.

16. What is a Test Case and what are its key components? How is it different from a Test Scenario? **[asked x4 in your bank]**

> **Answer:** A Test Case is a set of steps with test data, expected result and pass/fail criteria. Components: ID, title, preconditions, steps, test data, expected result, actual result, status. A Test Scenario is a high-level 'what to test'; one scenario gives many test cases.
>
> **Example:** Scenario: verify checkout with a coupon. Test cases: a valid coupon, an expired coupon, an invalid coupon, a coupon on a non-eligible item.

17. How do you prioritize test cases (especially when the release deadline is short)? **[asked x5 in your bank]**

> **Answer:** Prioritize by business impact, risk, usage frequency, defect history and recent changes. Run critical flows first, then high risk, then the rest if time allows.
>
> **Example:** With one day left: login, search, add to cart, payment and order confirmation first; wishlist colour themes later.

18. What are Entry and Exit Criteria in testing? **[asked x4 in your bank]**

> **Answer:** Entry criteria are the conditions that must be met before testing starts (stable build, environment ready, test cases reviewed, test data available). Exit criteria are conditions to stop (all critical cases executed, no open blocker/critical defects, agreed pass rate).
>
> **Example:** Entry: build deployed on QA with payment sandbox working. Exit: 95% pass, zero open critical defects in checkout.

19. What is the difference between Test Plan and Test Strategy?

> **Answer:** A Test Plan is project-specific: what, when, who and how for one release. A Test Strategy is an organisation or product-level, high-level approach (test types, tools, standards) that rarely changes.
>
> **Example:** Strategy: 'API and UI automation with Selenium, performance via JMeter'. Plan: this month's Diwali sale release schedule and scope.

20. What is a Traceability Matrix (RTM) and why is it important?

> **Answer:** RTM maps requirements to test cases (and defects) to make sure every requirement is covered and to support impact analysis when requirements change.
>
> **Example:** Requirement 'user can apply a coupon' maps to TC-45 to TC-52, and linked defects BUG-210.

21. What are Test Design Techniques? **[asked x2 in your bank]**

> **Answer:** Systematic techniques for creating effective test cases: Equivalence Partitioning, Boundary Value Analysis, Decision Table, State Transition, Use Case, Pairwise, and Error Guessing.
>
> **Example:** BVA on quantity (1 to 10) and a decision table for coupon plus membership discount combinations.

22. What is Equivalence Partitioning (EP)? Give an example.

> **Answer:** Divide inputs into valid and invalid groups where all values behave the same, and test one value from each group to reduce test cases.
>
> **Example:** Age field accepts 18-60: partitions are below 18 (invalid), 18-60 (valid), above 60 (invalid). Test 15, 30 and 65.

23. What is Boundary Value Analysis (BVA)? Give an example.

> **Answer:** Test at the edges of input ranges, where defects are most likely: min, min-1, min+1, max, max-1, max+1.
>
> **Example:** Cart quantity allows 1-10. Test 0, 1, 2, 9, 10 and 11.

Situational & Behavioral Scenarios (5)

24. Tell me about your recent testing project.

> **Answer:** Give a 1-minute structure: domain and application, your role, team size and methodology, modules you tested, types of testing, tools (JIRA, Postman, Selenium), and one achievement.
>
> **Example:** E-commerce platform: I tested the cart, checkout and payment modules in Agile sprints, and reduced production defects by adding coverage for payment edge cases.

25. What challenges have you faced during testing? **[asked x2 in your bank]**

> **Answer:** Name a real challenge and how you solved it: changing requirements, tight deadlines, unstable environments, hard-to-reproduce bugs. Show actions and the result.
>
> **Example:** Frequent requirement changes in the coupon module. I kept close to the PO, updated the test cases on the same day and automated the stable cases.

26. How do you test when requirements are unclear, missing or there is no documentation? **[asked x4 in your bank]**

> **Answer:** Clarify with the BA/PO/developers, review similar features and the previous behaviour, document my assumptions and get them confirmed, use exploratory testing and prototypes, and update the test cases as clarity improves.
>
> **Example:** For a new 'Buy Now Pay Later' option with no spec, I check how existing payment options behave, ask the PO about limits and refunds, and confirm by email.

27. How do you manage testing under tight deadlines or release pressure? **[asked x6 in your bank]**

> **Answer:** Prioritise by risk, test critical flows first, run smoke and sanity, automate repetitive checks, work in parallel with development, communicate risk clearly, and never silently skip testing.
>
> **Example:** Before a sale launch, I test checkout and payment fully, and sample-test low-risk pages like the 'About us' page.

28. How do you handle requirement changes during testing? **[asked x3 in your bank]**

> **Answer:** Assess the impact on existing test cases, update test cases and the RTM, tell the team about the effort or risk, retest and regress the affected areas, and ask for time or scope trade-offs.
>
> **Example:** Shipping charges change mid-sprint. I update the tax and shipping cases and regress cart, checkout and invoice.

Agile & Scrum (5)

29. What is Agile methodology? **[asked x2 in your bank]**

> **Answer:** Agile is an iterative and incremental approach that delivers working software in short cycles (sprints), with continuous customer feedback, collaboration and adaptation to change.
>
> **Example:** A 2-week sprint delivers 'Wishlist'. Feedback from users shapes the next sprint's 'Share wishlist' feature.

30. What are the ceremonies followed in Agile/Scrum? Describe each. **[asked x2 in your bank]**

> **Answer:** Sprint Planning (decide sprint scope), Daily Stand-up (15 min: yesterday, today, blockers), Sprint Review/Demo (show the work to stakeholders), Sprint Retrospective (what went well, what to improve), plus Backlog Grooming/Refinement.
>
> **Example:** Planning: pick 'coupon' stories. Stand-up: 'coupon API tested, blocked on test data'. Review: demo. Retro: 'improve test data availability'.

31. In a tight sprint, if a new requirement is added, how will you handle it?

> **Answer:** Assess the impact with the team and the PO. Either swap it with a story of equal size, or move it to the next sprint. I never absorb it silently; testing effort and risk must stay visible.
>
> **Example:** A new 'Gift wrap' story arrives on day 6. We swap it with a low-priority 'Sort by rating' story.

32. If the PO insists on releasing with a known bug, what will you do?

> **Answer:** I clearly present the risk, impact and evidence. The release decision belongs to the PO, so I make sure it is an informed one: document the known issue, agree on a workaround, and plan a fix in the next sprint or a hotfix.
>
> **Example:** A minor display issue in the order history is released with a known-issue note, but a payment failure would be escalated hard.

33. In Agile, what will you do if developers deliver a user story very late in the sprint?

> **Answer:** Raise it early, prioritise testing of the critical paths, use pair testing with developers, test in parallel where possible, and be transparent. If it cannot be done safely, move the story to the next sprint (Definition of Done is not negotiable).
>
> **Example:** The 'Apply coupon' story arrives on the last day. I test the main flow and negative cases, and carry the remaining checks to the next sprint if the DoD is not met.

Types & Levels of Testing (1)

34. What is Exploratory Testing and when do you use it? **[asked x3 in your bank]**

> **Answer:** Simultaneous learning, test design and execution without predefined scripts, guided by experience and curiosity, usually time-boxed with session notes. Used when requirements are weak, for new features, after scripted tests, and to find hidden defects.
>
> **Example:** A 45-minute session on the new checkout: try back button, double click on Pay, switching address mid-payment, and note issues found.

Testing Fundamentals, SDLC & STLC (2)

35. What is the difference between Verification and Validation? **[asked x4 in your bank]**

> **Answer:** Verification asks 'Are we building the product right?' (reviews, walkthroughs, static checks, without executing code). Validation asks 'Are we building the right product?' (executing the software to check it meets user needs).
>
> **Example:** Verification: reviewing the checkout requirement document. Validation: actually placing an order on the QA build.

36. How do you decide when to stop testing? How do you know your testing is good enough? **[asked x3 in your bank]**

> **Answer:** When exit criteria are met: planned test cases executed, critical/high defects closed, coverage and pass rate targets achieved, risks accepted by stakeholders, and the time or budget is exhausted. Testing is never 'complete', it is 'good enough for the risk'.
>
> **Example:** 100% of the checkout and payment cases executed, no open critical defects, and the PO signs off.

Scenario-Based Feature Testing (E-commerce & Real World) (2)

37. How would you test an E-commerce website (like Amazon or Flipkart)?

> **Answer:** Cover functional (search, cart, checkout, payment, orders, returns), UI/usability, compatibility (browsers, devices), performance (sale-day load), security (login, payment data), API/integration (payment gateway, inventory), and negative and edge cases.
>
> **Example:** Search 'shoes' > filter size > add to cart > apply coupon > pay by UPI > verify order email and stock reduction.

38. How would you test a login page beyond normal credentials?

> **Answer:** Test valid and invalid credentials, blank fields, case sensitivity, password masking, account lockout after failed attempts, SQL injection and XSS, session handling, 'remember me', forgot password, and browser back button after logout.
>
> **Example:** Enter ' OR 1=1-- as the username (should be rejected). Five wrong passwords should lock the account or show a captcha.

Section 2: High Priority (74 questions)

Asked frequently, mostly as process, scenario and 'how would you handle' questions. These separate an experienced tester from a fresher, so answer with a structured approach (steps, risks, trade-offs).

Defect Management & Bug Life Cycle (5)

39. How do you handle priority vs severity conflicts, e.g. business says a bug is minor but you think it is critical? **[asked x2 in your bank]**

> **Answer:** I explain the technical and user impact with evidence (data loss, revenue, number of users affected) and let the business make the final call in triage. I keep severity as an objective technical rating and accept that priority is a business decision.
>
> **Example:** Business calls a wrong tax display on invoices 'minor'. I show it is a legal and compliance risk, so it gets high priority.

40. How do you handle a bug that is hard to reproduce or occurs intermittently? Give an example. **[asked x2 in your bank]**

> **Answer:** Collect all context: exact steps, data, time, browser, logs, network and DB state. Try different environments and data, record the screen or session, add logging, and share the intermittent pattern with the developer. Mark it 'Intermittent' with the frequency.
>
> **Example:** The order sometimes shows 'Failed' even after payment. I found it only happens when the gateway callback is delayed more than 30 seconds, and proved it with logs.

41. How do you respond to "It's working on my machine" or "Not reproducible" from a developer or stakeholder? **[asked x2 in your bank]**

> **Answer:** Stay calm and collaborative: share environment, build, data, video and logs; compare configurations; try on their machine or a shared environment. Many differences come from data, config, cache or version.
>
> **Example:** A coupon works for the developer but fails for me. The difference is that the coupon is user-specific and my test account was not eligible.

42. What is Bug Triage and how is it conducted? **[asked x3 in your bank]**

> **Answer:** A meeting where new defects are reviewed and their validity, severity, priority, owner and target release are decided. Attendees: QA, Dev lead, PO/BA, sometimes the Scrum Master. Done regularly (daily in a busy phase, or weekly).
>
> **Example:** Triage decides that a payment failure is P1 for this sprint, and a banner misalignment is P3 for later.

43. What is Defect Leakage (Escape Defect)? How do you reduce it? **[asked x2 in your bank]**

> **Answer:** A defect that escapes testing and is found in production (or the next phase). Reduce it through better requirement review, risk-based coverage, RCA of each leak, adding regression cases, exploratory testing and a realistic test environment and data.
>
> **Example:** A leaked bug: the discount is applied twice on a returning order. I add that case to the suite and review similar flows.

Smoke / Sanity / Regression & Automation Decisions (8)

44. What is Regression Testing and why is it important?

> **Answer:** Re-running existing tests after a change (fix, new feature, config) to confirm nothing that previously worked is broken. It protects stable features and is a good candidate for automation.
>
> **Example:** After adding a new payment option, I regress cart, other payment modes, order history and refunds.

45. What is the difference between Functional and Regression testing?

> **Answer:** Functional testing verifies the feature against requirements (a new or changed feature). Regression testing verifies that existing features still work after a change.
>
> **Example:** Functional: test the new 'Gift card' payment. Regression: confirm card, UPI and COD payments still work.

46. How many test cases are in your regression suite and how long does it take to execute?

> **Answer:** Answer with your project's real numbers: for example, 'around 600 cases, 350 automated in about 1.5 hours, and the manual part in 2 days, with a critical subset of 80 cases for quick builds'.
>
> **Example:** Say which modules the suite covers (cart, checkout, payment, orders) and how you reduced time over the previous releases.

47. Element IDs change frequently, causing locator brittleness. How do you handle it?

> **Answer:** Ask developers to add stable test IDs/attributes, use robust locators (relative XPath, CSS with stable attributes, not index-based), apply the Page Object Model to keep locators in one place, and use explicit waits.
>
> **Example:** The 'Add to Cart' button ID changes every build. I use a data-testid='add-to-cart' attribute and keep it in one page class.

48. File upload intermittently fails due to browser/OS dialog or security policy in Chrome/Edge. How do you handle it?

> **Answer:** Avoid the native dialog: send the file path directly to the input[type=file] element (sendKeys), instead of clicking through the OS dialog. Keep browser and driver versions aligned, check for the security/download policies, and use a stable test file location.
>
> **Example:** Profile picture upload: send the image path to the file input and verify the preview, instead of automating the Windows dialog.

49. Your Pull Request (PR) for automation is not approved for merging. How do you handle it / convince the developer? **[asked x2 in your bank]**

> **Answer:** Listen to the review comments, understand the standards (naming, structure, reusability), explain my design with data, and fix valid points quickly. I treat it as a learning and alignment exercise, not a fight.
>
> **Example:** The reviewer says my locators are duplicated. I move them to a page class, and then the PR is approved.

50. When do you automate: in the current sprint or the next sprint?

> **Answer:** Ideally in the same sprint once the feature is stable (shift-left), so that regression is ready for the next release. If the feature is still changing or the time is short, automate in the next sprint, and keep the manual testing in the current one.
>
> **Example:** A stable login/checkout flow is automated in the sprint. A new UI under redesign is automated next sprint.

51. What is the difference between Manual Testing and Automation Testing?

> **Answer:** Manual is human-driven, flexible, good for exploratory, usability and ad-hoc testing, but slower and error-prone for repetition. Automation uses scripts and tools, is fast and repeatable and good for regression and data-driven tests, but has setup and maintenance costs.
>
> **Example:** Manual: check how the new product page looks and feels. Automation: run the checkout regression with 50 payment/data combinations nightly.

Test Design, Planning & Documentation (8)

52. What is the difference between Boundary Value Analysis and Equivalence Partitioning?

> **Answer:** EP divides inputs into classes and picks one value from each class. BVA focuses on the edges of those classes. They work together: EP to reduce the cases, BVA to catch the edge defects.
>
> **Example:** Quantity 1-10: EP picks 5 (valid), -1 and 15 (invalid). BVA picks 0, 1, 10 and 11.

53. What is Risk-Based Testing?

> **Answer:** Prioritising testing based on the probability and impact of failure. High-risk areas get deeper and earlier testing; low-risk areas get lighter coverage.
>
> **Example:** Payments and order placement are high risk (money, trust), so they get thorough testing. The 'About us' page gets a quick check.

54. How do you ensure your test cases are effective?

> **Answer:** By reviewing them against requirements (RTM), applying design techniques (EP/BVA), including negative cases, peer review, tracking defects found per test case, and updating the cases after each production leak.
>
> **Example:** A review finds that the coupon cases lack an 'expired coupon' case, so I add it.

55. How do you identify redundant test cases?

> **Answer:** Review the suite for the same steps and same expected results, check the RTM for requirements with many overlapping cases, merge or remove duplicates, and keep one case for each unique condition.
>
> **Example:** Three cases check 'login with a valid user'. I merge them into one and keep the different data combinations as data sets.

56. Explain Decision Table Testing with an example.

> **Answer:** A table of conditions (inputs) and the resulting actions (outputs), covering every combination of rules. Useful when many business rules combine.
>
> **Example:** Free shipping rule: Order > Rs.499 AND Prime member. Combinations: (Y,Y) free, (Y,N) free, (N,Y) free, (N,N) paid. Each becomes a test case.

57. What is State Transition Testing? Give an example.

> **Answer:** Testing a system that changes state based on events or conditions, verifying valid and invalid transitions.
>
> **Example:** Order states: Placed > Packed > Shipped > Delivered. Valid: Placed > Cancelled. Invalid: Delivered > Packed.

58. What are the key components of a test strategy document?

> **Answer:** Scope and objectives, test approach and levels, test types, tools and automation approach, environment and data, defect management and reporting, roles, entry/exit criteria, risks, and metrics.
>
> **Example:** The strategy states: UI and API automation for regression, manual exploratory for new features, and JMeter for the sale-day load.

59. What is a Test Summary Report?

> **Answer:** A document summarising testing: scope, test cases executed/passed/failed, defect counts by severity and status, open risks, coverage, and a go/no-go recommendation.
>
> **Example:** Release 5.2: 520 cases executed, 96% pass, 2 medium defects open (known issues), recommendation: Go.

Situational & Behavioral Scenarios (4)

60. How do you test when the test environment is unstable? **[asked x2 in your bank]**

> **Answer:** Raise and document environment issues, use a smoke test after each deployment, keep environment downtime visible, use mocks/stubs for unstable dependencies, shift tests to a stable environment, and log the impact on the plan.
>
> **Example:** The payment sandbox is down, so I use a mock payment service to continue the cart/order tests.

61. Have you ever disagreed with a developer? How do you handle conflict with a developer? **[asked x2 in your bank]**

> **Answer:** Yes, usually over the validity or priority of a defect. I focus on facts, share evidence, listen to their view, involve the BA/PO if needed, and keep it professional. The aim is the product's quality, not winning.
>
> **Example:** Developer says the discount rounding is fine. I show the invoice figure differs by Rs.1 from the cart, and we align with the PO.

62. How do you convince management to give more time for testing?

> **Answer:** Present data and risk: past defect leakage, scope growth, number of test cases and the effort required, the cost of a production failure, and a prioritised plan showing what will and will not be covered in the time given.
>
> **Example:** I show that the payment module needs 3 more days, and that a production failure on sale day costs far more.

63. How do you test with limited test data?

> **Answer:** Prioritise critical scenarios, reuse and reset data, create data through the API/scripts/DB, use data masking from production, apply EP/BVA to cut the number of values, and share data across tests safely.
>
> **Example:** Create a few users (new, existing, blocked) and reuse them, with a script that resets cart and orders before each run.

Agile & Scrum (10)

64. What is the difference between Agile and Waterfall?

> **Answer:** Waterfall is sequential and plan-driven, with a fixed scope and late testing. Agile is iterative with short sprints, continuous testing and feedback, and welcomes changes.
>
> **Example:** Waterfall: the whole e-commerce site is delivered after months. Agile: cart in sprint 1, wishlist in sprint 2, and so on.

65. What are the Scrum roles (PO, Scrum Master, Dev Team)? Who is your Scrum Master? **[asked x2 in your bank]**

> **Answer:** Product Owner owns the backlog and priorities. Scrum Master facilitates the process and removes blockers. Development Team (including QA) builds and tests the product. Answer the 'who is your Scrum Master' part with your actual name.
>
> **Example:** PO decides 'coupons' go first, the SM removes the blocker for the test environment, and the team delivers.

66. What is Agile Testing?

> **Answer:** Testing practised in Agile: continuous, collaborative, starts early with the requirement and runs inside the sprint. QA works with developers and the PO, with a focus on fast feedback and automation.
>
> **Example:** QA reviews the 'Apply coupon' story at refinement, tests it as soon as the dev completes it, and automates it in the same sprint.

67. What is a User Story? **[asked x2 in your bank]**

> **Answer:** A short requirement from the user's view: 'As a <user>, I want <goal>, so that <benefit>', with acceptance criteria.
>
> **Example:** As a shopper, I want to save items to a wishlist so that I can buy them later.

68. What is Acceptance Criteria? How do you test a story when the AC is unclear? **[asked x3 in your bank]**

> **Answer:** Acceptance Criteria are the conditions a story must meet to be accepted. If they are unclear, I ask the PO/BA to clarify in refinement, write my understanding as examples (Given/When/Then), confirm them, and then test.
>
> **Example:** AC says 'wishlist limit applies'. I ask 'what is the limit and what error is shown?' before testing.

69. What is the Definition of Done (DoD)? **[asked x2 in your bank]**

> **Answer:** A shared checklist that a story must meet to be called done: coded, reviewed, tested, defects fixed, regression passed, documented and demo-ready.
>
> **Example:** 'Wishlist' is done only when its test cases pass, no open critical bugs remain, and automation is merged.

70. What is a Product Backlog? How is it different from a Sprint Backlog? **[asked x3 in your bank]**

> **Answer:** The Product Backlog is the full, prioritised list of everything the product may need, owned by the PO. The Sprint Backlog is the subset of stories the team commits to for the current sprint, with its tasks.
>
> **Example:** Product backlog: wishlist, coupons, gift cards, reviews. Sprint backlog: wishlist and coupons for this sprint.

71. What is velocity in a sprint and how is it tracked? **[asked x2 in your bank]**

> **Answer:** Velocity is the amount of work (story points) a team completes in a sprint. It is tracked as the average of the past sprints and used for forecasting.
>
> **Example:** The team completes 30, 28 and 32 points in 3 sprints, so velocity is about 30, and the next sprint is planned for about 30 points.

72. How do you estimate testing effort in Agile?

> **Answer:** Based on the story size and complexity: number of test scenarios, automation work, regression impact, test data and environment needs, and past velocity. Use story points or hours in planning poker with the team.
>
> **Example:** The coupon story is a 5 pointer: 12 test cases, 1 day for automation and 0.5 day for regression.

73. How do you handle a blocker bug in the middle of a sprint?

> **Answer:** Report it immediately, discuss it in the stand-up, check with the PO/Scrum Master on priority, swap testable work, keep testing the unblocked areas and, if needed, re-plan the sprint scope.
>
> **Example:** A login blocker stops all checkout testing, so I test the guest flow and the API while Dev fixes it.

Types & Levels of Testing (11)

74. What is the difference between Exploratory and Ad-hoc Testing? **[asked x2 in your bank]**

> **Answer:** Exploratory testing is structured and time-boxed, with a charter and notes, where the tester learns and designs tests as they go. Ad-hoc testing is informal and unplanned, with no documentation, usually based on random checks.
>
> **Example:** Exploratory: a 1-hour charter 'explore checkout with interrupted payments'. Ad-hoc: randomly clicking around the site before the demo.

75. What is Ad-hoc Testing? **[asked x2 in your bank]**

> **Answer:** Informal and unplanned testing without test cases or documentation, relying on the tester's knowledge and intuition to find defects quickly.
>
> **Example:** Quickly changing the quantity to a big number, using special characters in the address, and pressing back during payment.

76. What are the different levels of testing?

> **Answer:** Unit, Integration, System and Acceptance (UAT) testing.
>
> **Example:** Unit: the discount function. Integration: cart with the payment gateway. System: the whole site. UAT: business users validate an order end to end.

77. What is Integration Testing? What are its types?

> **Answer:** Testing the interaction between modules or systems to find interface and data-flow defects. Types: Big Bang, Top-Down, Bottom-Up, and Sandwich (Hybrid).
>
> **Example:** Cart + inventory + payment: after payment success, stock reduces and the order is created.

78. What is White-Box Testing and how is it different from Black-Box Testing? **[asked x2 in your bank]**

> **Answer:** White-box testing checks internal code, logic and paths (done by developers/with code knowledge). Black-box testing checks behaviour against requirements with no code knowledge (done by QA).
>
> **Example:** White-box: verify the discount calculation branches. Black-box: apply a coupon on the UI and check the total.

79. What is End-to-End (E2E) Testing? How is it different from System Testing?

> **Answer:** E2E testing validates a complete business flow across all connected systems from start to finish. System testing validates the complete integrated application against requirements, often module by module.
>
> **Example:** E2E: search > cart > pay > order email > shipment > refund. System: test the cart and payment modules separately in the full system.

80. What is User Acceptance Testing (UAT)?

> **Answer:** Final validation by business users or the client to confirm the product meets business needs and is ready to go live.
>
> **Example:** The business team places real-world orders with coupons and refunds on the staging site before sign-off.

81. What is Non-Functional Testing? How is it different from Functional Testing?

> **Answer:** Non-functional testing checks how the system performs: speed, load, security, usability, compatibility and reliability. Functional testing checks what the system does against requirements.
>
> **Example:** Functional: does the order get placed? Non-functional: does it place within 3 seconds with 5,000 users?

82. What is Performance Testing?

> **Answer:** Testing how fast, stable and scalable the system is under expected workload (response time, throughput, resource usage).
>
> **Example:** Search results should load within 2 seconds with 2,000 concurrent shoppers.

83. What is Load Testing?

> **Answer:** Testing the system under expected and peak user load to verify response times and stability.
>
> **Example:** 1,000 users add to cart and check out at the same time, as on a normal sale day.

84. What is Stress Testing?

> **Answer:** Testing beyond the maximum load to see how and when the system breaks and how it recovers.
>
> **Example:** Push traffic to 10x the usual sale traffic and check that the site fails gracefully and recovers when load drops.

Testing Fundamentals, SDLC & STLC (1)

85. What is the Software Testing Life Cycle (STLC)? Explain its phases. **[asked x3 in your bank]**

> **Answer:** Phases: Requirement Analysis, Test Planning, Test Case Development, Test Environment Setup, Test Execution, and Test Closure. Each has entry and exit criteria and its deliverables.
>
> **Example:** Requirement analysis of the 'coupon' story > plan > write cases > set up QA > execute > closure report.

Scenario-Based Feature Testing (E-commerce & Real World) (14)

86. How many test cases will you write to test an E-commerce site?

> **Answer:** No fixed number; it depends on scope, risk and priority. I estimate through modules and features: for example, scenarios per module, then cases per scenario with EP/BVA, covering positive, negative and edge cases. Mention a sensible figure only as a rough range, for example, 500-1,000 for a full site.
>
> **Example:** Cart: 40, checkout: 60, payments: 80, search: 50, login/registration: 50, and so on.

87. How do you test “Add to Cart” functionality in an e-commerce app?

> **Answer:** Verify adding single and multiple items, quantity update, remove item, cart persistence (login/logout, devices), stock limits, price and total calculation, out-of-stock items, maximum quantity, and the cart count badge.
>
> **Example:** Add 2 shirts: total = 2 x price. Update quantity to 11 where the limit is 10: expect an error message.

88. How do you test the checkout process with multiple payment methods?

> **Answer:** Test each method (card, UPI, net banking, wallet, COD, EMI) for success, failure, cancel, timeout and retry. Verify the amount, order status, payment status, stock, and confirmation email/SMS, plus security (no card data in logs).
>
> **Example:** UPI payment succeeds: order is Confirmed, and a failed payment keeps the cart items and allows a retry.

89. How do you ensure test cases cover edge scenarios for a payment gateway?

> **Answer:** Consider declined card, insufficient funds, expired card, wrong CVV/OTP, network drop mid-payment, double-click on Pay, gateway timeout, duplicate transaction, partial payment, currency mismatch and callback delay.
>
> **Example:** Close the browser after money is debited. The order should still be created or auto-refunded, never a silent loss.

90. How do you test search functionality with filters (price, brand, category)? **[asked x2 in your bank]**

> **Answer:** Check the result relevance, each filter alone and combined, price range boundaries, sorting, pagination, the result count, clearing filters, the 'no results' state, special characters, and speed.
>
> **Example:** Search 'sneakers', filter brand = Nike and price Rs.2,000-4,000: every result must satisfy both filters.

91. How do you test product price calculation (discounts, tax, shipping)?

> **Answer:** Verify the formula: item price - discount + tax + shipping, for different tax rates, quantities, rounding, coupon stacking, regions and currencies. Compare the cart, checkout, invoice and email totals.
>
> **Example:** Price Rs.1,000, 10% discount, 18% GST, Rs.50 shipping: Rs.900 + Rs.162 + Rs.50 = Rs.1,112.

92. How do you test “Apply Promo Code” / coupons and discount codes? **[asked x2 in your bank]**

> **Answer:** Test a valid code, invalid, expired, already used, minimum cart value, product/category restrictions, case sensitivity, stacking with other offers, removal of the coupon and the total recalculation.
>
> **Example:** SAVE10 needs a minimum Rs.999: with Rs.900 cart, it must show a clear error.

93. How do you test the refund flow and refund timeline in e-commerce? **[asked x2 in your bank]**

> **Answer:** Verify refund eligibility, partial and full refund, refund to the original payment method, amounts (including shipping/tax), status updates, notification emails and the stated timeline (for example, 5-7 days), and the report entries.
>
> **Example:** Return approved for a Rs.1,500 order: refund status moves from Initiated to Completed, and the amount is credited to the same card.

94. How do you test user registration with email and mobile number?

> **Answer:** Test valid and invalid email/mobile formats, duplicates, OTP verification (valid, wrong, expired, resend), password rules, mandatory fields, terms checkbox, confirmation email, and SQL/XSS inputs.
>
> **Example:** A registration with an existing mobile number should say 'already registered'.

95. How do you test "Forgot Password" functionality?

> **Answer:** Verify the reset link/OTP is sent only to registered users, the link expires and works once, password rules are applied, the old password stops working, no user enumeration in messages, and security (rate limiting).
>
> **Example:** Use the reset link a second time. It should say the link is expired or used.

96. How do you test an OTP-based login system?

> **Answer:** Test valid OTP, wrong OTP, expired OTP, resend (old OTP invalidated), attempt limit and lockout, OTP delivery delay, auto-read on mobile, and that the OTP is not exposed in the response or logs.
>
> **Example:** After 3 wrong OTPs, the user is blocked for some minutes. An OTP older than 5 minutes should be rejected.

97. How do you test cross-browser compatibility? **[asked x2 in your bank]**

> **Answer:** Run the same critical flows on supported browsers and versions (Chrome, Firefox, Safari, Edge) and operating systems, using BrowserStack/Selenium Grid. Check layout, functionality, JavaScript and CSS differences.
>
> **Example:** Checkout works in Chrome, but the Pay button is hidden in Safari. Raise it with a screenshot.

98. How do you test mobile responsiveness of an e-commerce site?

> **Answer:** Test on different screen sizes and orientations, real devices and emulators, touch targets, menus, images, text wrapping, keyboard type for fields (numeric for mobile), and performance on slow networks.
>
> **Example:** On a 360px-wide phone, the product grid shows 1-2 columns and the Buy button is fully visible.

99. How do you test concurrency when multiple users access the same product stock or update the cart at the same time? **[asked x2 in your bank]**

> **Answer:** Simulate parallel actions with two sessions or tools (JMeter or scripts) and check the stock count, no overselling, correct cart state, locking, and clear error messages. Verify the final data in the DB.
>
> **Example:** Two users buy the last unit at the same time: one order succeeds and the other gets 'out of stock'.

Integration, DB, API, Performance & Security Scenarios (5)

100. What if an API request passes but the UI fails (e.g. Order API says "Success" but UI shows "Failed")?

> **Answer:** Investigate where it breaks: API response, UI mapping, error handling, caching or timing. Compare the API payload with what the UI shows, check the console and network logs, and raise it to the right team with evidence. It is often a front-end handling or data-mapping defect.
>
> **Example:** The Order API returns 200 with status 'SUCCESS', but the UI reads the wrong field, so it shows 'Failed'. A front-end bug.

101. How do you ensure data consistency across UI, API and DB (e.g. DB data integrity after placing an order)? **[asked x2 in your bank]**

> **Answer:** Compare the same record across the layers: UI display, API response and DB values (order, payment, stock, address). Check totals, statuses and timestamps, and verify there are no duplicate or missing records.
>
> **Example:** After an order, the UI shows total Rs.1,112, the API returns 1112, and the DB orders table matches, with stock reduced by 1.

102. How do you test security in login (brute force, SQL injection)? **[asked x2 in your bank]**

> **Answer:** Test brute force (lockout/captcha after failed attempts), SQL injection and XSS in fields, password encryption, HTTPS, session/cookie flags, session expiry and no sensitive data in URLs or error messages.
>
> **Example:** Enter admin' -- in the username. The system must not log in or reveal a database error.

103. How do you test performance for a web app?

> **Answer:** Define goals (response time, throughput), build realistic user scenarios, run load/stress/endurance tests with tools such as JMeter, monitor server resources, find bottlenecks, and retest after fixes.
>
> **Example:** Test 5,000 users on search and checkout, aiming for under 3 seconds and an error rate below 1%.

104. How do you test without a UI (back-end or API-only features)? **[asked x2 in your bank]**

> **Answer:** Use API tools such as Postman or REST Assured: verify status codes, response body, headers, schema, validations, authentication, error handling, and then confirm the data in the DB or logs.
>
> **Example:** POST /orders with a valid payload returns 201 and creates the DB record. With an invalid quantity, it returns 400 with a message.

Metrics, Coverage & Reporting (5)

105. What are Test Metrics? **[asked x2 in your bank]**

> **Answer:** Quantitative measures of testing progress and quality, such as test cases executed/passed/failed, defect density, defect leakage, pass percentage, automation coverage and defect removal efficiency.
>
> **Example:** Release report: 95% executed, 92% passed, 4 open defects, 2% leakage.

106. How do you report testing progress (to management / in Agile)? **[asked x2 in your bank]**

> **Answer:** Through daily stand-up updates, dashboards (JIRA, execution status), the test summary report and sprint review, covering executed vs planned, defects by severity, blockers, risks and the release recommendation.
>
> **Example:** A short daily update: 'Checkout 80% done, 2 blockers, payment testing starts tomorrow'.

107. How do you ensure maximum test coverage in less time? What is Test Coverage? **[asked x3 in your bank]**

> **Answer:** Test coverage measures how much of the requirement or code is tested. Maximise it in less time by risk-based prioritisation, EP/BVA, automation, an RTM to find gaps, and exploratory testing in the remaining areas.
>
> **Example:** I cover every requirement for payments, and the less-used pages only by smoke tests.

108. How do you ensure 100% test coverage?

> **Answer:** 100% coverage is practically impossible (exhaustive testing is not feasible). I aim for 100% requirement coverage using an RTM and high-risk code/path coverage, and accept risk-based coverage elsewhere.
>
> **Example:** Every requirement of 'Checkout' maps to at least one test case, but I will not test every possible address and cart combination.

109. What KPIs do you show to stakeholders?

> **Answer:** Pass/fail percentage, defect leakage, defect density, defect removal efficiency, automation coverage, requirement coverage, execution progress and open critical defects. Choose those that help decisions, not vanity numbers.
>
> **Example:** Show that checkout defect leakage fell from 6% to 2% over three releases.

JIRA & Defect Tracking Tools (3)

110. What is JIRA and why do QA engineers use it?

> **Answer:** JIRA is a project and issue-tracking tool for Agile teams. QA uses it for bug tracking, linking tests to stories, boards and sprint tracking, dashboards and reports.
>
> **Example:** A bug 'Coupon not applied' is logged in JIRA, linked to the 'Coupon' story and assigned to the developer.

111. How do you create a Bug in JIRA?

> **Answer:** Click Create, select the project and the issue type 'Bug', and fill in the summary, description with steps, expected and actual result, environment, severity/priority, affects version, assignee and attachments, and link it to the story. Then submit.
>
> **Example:** Summary: 'Order total mismatch after coupon', with steps, a screenshot, and linked to the story WISH-45.

112. How do QA engineers use JIRA for Agile testing?

> **Answer:** QA uses JIRA to see stories on the sprint board, link test cases and bugs to stories, track status through the workflow, use dashboards/filters for progress, and join sprint reports and retrospectives.
>
> **Example:** The board shows the 'Wishlist' story moving from In Dev to In QA to Done, with its linked bugs.

Section 3: Medium Priority (92 questions)

Breadth topics. Asked occasionally, usually as follow-ups to your earlier answers or to probe a specific area on your resume. One clear paragraph per question is enough.

Defect Management & Bug Life Cycle (6)

113. What is a Deferred bug?

> **Answer:** A valid defect that is postponed to a future release because of low priority, low impact or time/cost constraints, with the PO's agreement.
>
> **Example:** A minor alignment issue on the wishlist page is deferred to the next release.

114. What are Duplicate and Invalid bugs? How do you handle them?

> **Answer:** A Duplicate is a defect already reported by someone else. An Invalid bug is not a real defect (wrong understanding, expected behaviour, or an environment/data issue). I search JIRA before raising, link duplicates to the original and close them, and learn from invalid ones.
>
> **Example:** Two testers log 'Pay button not working'. I link the second to the first and close it as Duplicate.

115. What is Root Cause Analysis (RCA)?

> **Answer:** Finding the underlying cause of a defect (not just the symptom) so that it can be prevented in the future, using methods such as 5 Whys.
>
> **Example:** Wrong discount in production: the cause was a missed requirement update, so we add a requirement review step.

116. What is a Showstopper Defect?

> **Answer:** A defect so severe that it blocks further testing or release (crash, data loss, or a core function not working).
>
> **Example:** The checkout page crashes, so no order can be placed.

117. What is Defect Density?

> **Answer:** The number of defects per size of the software (per KLOC, function point or module), used to judge the quality of a module.
>
> **Example:** The payment module has 12 defects in 4 KLOC, i.e., density 3 per KLOC.

118. What is Defect Clustering?

> **Answer:** A small number of modules contain most of the defects (the Pareto principle, roughly 80/20), so those areas deserve more testing.
>
> **Example:** Most bugs come from the payment and coupon modules, so I test them deeper.

Smoke / Sanity / Regression & Automation Decisions (3)

119. How do you test automation scripts in parallel execution?

> **Answer:** Make the scripts independent (own data, no shared state), use thread-safe drivers, run on Selenium Grid or the cloud with TestNG/JUnit parallel settings, and analyse the results per thread.
>
> **Example:** Run login, search and cart tests in parallel on Chrome and Firefox, each with its own user.

120. What is Build Acceptance Testing (BAT)?

> **Answer:** A quick set of critical tests (same as smoke) run on a new build to decide whether it is acceptable for deeper testing.
>
> **Example:** Login, search, add to cart and payment page loading on the new build.

121. Given test cases with priority -1, 0, 1, 2, what is the sequence of execution?

> **Answer:** Normally the lowest number is executed first when the numbering represents the order (-1, 0, 1, 2), so -1 runs first and 2 last. Confirm the convention in your tool/project (in some tools, 1 is the highest priority).
>
> **Example:** Priority -1 (checkout flow) runs first, then 0 (search), 1 (wishlist) and 2 (UI polish).

Test Design, Planning & Documentation (4)

122. What is Use Case Testing?

> **Answer:** Designing tests from use cases, which describe the actor's interactions with the system, including the main flow, alternate flows and exceptions.
>
> **Example:** Use case 'Place order': main flow (pay and confirm), alternate flow (use a coupon), exception (payment fails).

123. What is Error Guessing?

> **Answer:** Using experience and intuition to guess where defects may be hidden, and writing tests for them.
>
> **Example:** Try an empty cart checkout, a negative quantity, or a name with special characters.

124. What is Pairwise Testing?

> **Answer:** A technique that tests all pairs of parameter values rather than every combination, to reduce the number of tests while still finding most defects.
>
> **Example:** Browser x OS x payment method: use pairs to cover the combinations in 12 tests instead of 60.

125. How did you design test cases for CSV file validation (format, delimiter, data integrity)?

> **Answer:** Validate the file format and extension, header and column names, delimiter, mandatory fields, data types, lengths, special characters, duplicates, empty rows, large file size, encoding (UTF-8) and the data loaded into the DB.
>
> **Example:** Upload a product CSV with a missing price column, a comma inside a name, and 10,000 rows, and check errors and imported data.

Situational & Behavioral Scenarios (1)

126. If two developers raise bugs on each other’s code, how do you handle it as QA?

> **Answer:** Stay neutral: reproduce the bug, find the root cause with logs, and show the evidence to both. If needed, involve the tech lead. The aim is the fix, not blame.
>
> **Example:** Dev A says the cart bug is in the API, and Dev B says it is the UI. I show the API response is correct, so the UI owner fixes it.

Agile & Scrum (19)

127. What is the Sprint Retrospective? What is your role in it as QA? **[asked x3 in your bank]**

> **Answer:** The end-of-sprint meeting where the team discusses what went well, what did not and the improvement actions. As QA, I share testing blockers (late builds, test data issues), suggest process improvements and take actions for my area.
>
> **Example:** I raise 'stories arrive too late for testing' and suggest a mid-sprint build handover.

128. What is the purpose of Sprint Planning?

> **Answer:** To decide what the team will deliver in the sprint, clarify the stories and AC, estimate and commit. QA raises testability questions and effort.
>
> **Example:** QA points out that the coupon story needs test data and extra time for regression.

129. What is the Daily Stand-up?

> **Answer:** A 15-minute daily meeting where everyone shares yesterday's work, today's plan and blockers.
>
> **Example:** 'Yesterday I tested the cart API, today I test checkout, and I am blocked by the payment sandbox'.

130. What is the Sprint Review?

> **Answer:** The meeting at the end of the sprint where the team demos completed work to the stakeholders and collects feedback.
>
> **Example:** The team demonstrates the working wishlist, and the PO asks for a 'share wishlist' option.

131. How does Story Points estimation work?

> **Answer:** Teams assign relative size points (often Fibonacci) to stories considering complexity, effort and uncertainty, usually by planning poker, and use them for velocity.
>
> **Example:** 'Add to wishlist' = 3 points, 'Coupon engine' = 8 points.

132. What are Burndown charts?

> **Answer:** Charts showing the remaining work against time in a sprint, to see whether the team is on track to finish.
>
> **Example:** If the line stays flat for 3 days, we are behind and take action.

133. What is the Agile Testing Pyramid?

> **Answer:** A strategy with many fast unit tests at the base, fewer API/service tests in the middle, and the fewest slow UI tests at the top.
>
> **Example:** Unit tests for the discount logic, API tests for the order service, and a few UI tests for the checkout flow.

134. What is Sprint Testing?

> **Answer:** Testing the stories inside the sprint as soon as they are ready, along with development, so that feedback is quick and the stories meet the DoD.
>
> **Example:** Test the coupon story on day 4 of the sprint, and not after the sprint ends.

135. What is Continuous Integration in Agile?

> **Answer:** Developers merge code frequently into a shared repository, and each merge automatically builds and runs tests, to find integration issues early.
>
> **Example:** Each commit triggers a build and runs the smoke and unit tests in Jenkins.

136. What is Shift-Left Testing?

> **Answer:** Moving testing activities earlier in the lifecycle (requirement reviews, unit tests, static checks) to find defects when they are cheaper to fix.
>
> **Example:** QA reviews the 'coupon' story and finds a missing rule on 'stacking' before coding starts.

137. You have 30+ sprints in your release. How will you design and run your test scripts?

> **Answer:** Keep a living, modular regression suite, automate the stable features, tag tests (smoke/regression), maintain the RTM, run smoke on every build and a full regression per release, and retire obsolete cases.
>
> **Example:** Each sprint adds its new stable cases to the regression pack, and the nightly run covers all of it.

138. If you find a bug during the sprint demo, how do you handle it?

> **Answer:** Note the issue calmly, confirm the behaviour, log it as a defect and tell the PO and team at once. Decide whether to fix it now or defer it, and learn why it was missed in testing.
>
> **Example:** During the demo the coupon field accepts spaces. I log it and add the case to the suite.

139. How do you test a user story that has dependencies on another team?

> **Answer:** Use mocks/stubs or contract testing for the dependency, agree on API contracts early, test your side thoroughly, and plan integration testing as soon as the other team delivers, and track the dependency as a risk.
>
> **Example:** The payment team's API is not ready, so I test checkout against a mock response and integrate later.

140. How do you test bug fixes quickly in Agile?

> **Answer:** Run a quick retest of the fix, then focused regression of the impacted area, using automation and smoke checks, and keep clear notes in the defect.
>
> **Example:** After the coupon fix, I run the coupon cases and the automated cart regression within an hour.

141. How do you manage test cases in Agile where requirements evolve?

> **Answer:** Keep test cases modular and linked to stories, review them every sprint, update or retire those affected by changes, and keep the RTM current. Use exploratory charters where requirements are still moving.
>
> **Example:** When the shipping rule changes, I update the 6 related cases the same day.

142. How do you test Agile user stories that don't fit in one sprint?

> **Answer:** Split the story into smaller testable slices, test each slice as it is done, use feature flags, and plan integration and regression at the point where the story completes.
>
> **Example:** The 'Return and Refund' epic is tested in slices: return request, approval, refund, notification.

143. How do you test when development and QA environments differ?

> **Answer:** Document the differences (versions, data, configuration), try to align the environments, keep a checklist of known gaps, and run smoke tests on each environment. Validate the critical paths on a production-like environment.
>
> **Example:** QA has a mock payment gateway while staging has the real sandbox, so I rerun payment tests on staging.

144. How do you test when multiple teams deliver microservices in parallel?

> **Answer:** Use contract tests, API tests for each service, mocks for unavailable services, an integration environment with agreed versions, and end-to-end tests for critical flows.
>
> **Example:** Cart, inventory and payment services each have API tests, and one E2E test covers the full order.

145. How do you handle resource allocation in Agile QA?

> **Answer:** Plan by risk and workload, share the work by skills (manual, automation, API), pair up on complex stories, cross-train, and make bottlenecks visible during planning.
>
> **Example:** A new joiner pairs with me on the cart tests while I focus on payments.

Types & Levels of Testing (12)

146. What are the types of testing?

> **Answer:** Broadly: Functional (unit, integration, system, UAT, smoke, sanity, regression) and Non-functional (performance, security, usability, compatibility). By approach: manual vs automation, and black-box, white-box and grey-box.
>
> **Example:** Functional: checkout works. Non-functional: checkout is fast, secure and works on mobile.

147. What is Functional Testing?

> **Answer:** Testing the application's features against the functional requirements, by giving inputs and checking outputs.
>
> **Example:** Add an item to the cart, then verify the item, quantity and total.

148. What is System Testing?

> **Answer:** Testing the complete, integrated application against the specified requirements, as a whole.
>
> **Example:** Test the whole site on the QA environment: register, search, buy and receive the email.

149. What is Usability Testing and why is it important? **[asked x2 in your bank]**

> **Answer:** Checking how easy, clear and pleasant the product is for real users. It matters for conversion and customer satisfaction.
>
> **Example:** Can a new user find the coupon field and complete checkout in fewer than 5 steps?

150. What is Security Testing?

> **Answer:** Testing that the application protects data and resists attacks: authentication, authorisation, input validation, encryption and session handling.
>
> **Example:** A user must not see another user's orders by changing the order ID in the URL.

151. What is Compatibility Testing?

> **Answer:** Verifying that the application works across browsers, operating systems, devices, screen sizes and network conditions.
>
> **Example:** Checkout works on Chrome, Safari, Android and iOS.

152. What is Localization Testing?

> **Answer:** Verifying the application is correct for a region: language, currency, date/time format, address format and cultural content.
>
> **Example:** The Indian site shows Rs., the dd/mm/yyyy date and Hindi labels correctly.

153. What is Accessibility Testing and why is it important? **[asked x2 in your bank]**

> **Answer:** Checking that people with disabilities can use the application (screen readers, keyboard navigation, contrast, alt text), following WCAG. It matters for inclusion and legal compliance.
>
> **Example:** A screen reader reads the product name and the 'Add to cart' button, and the page works with the keyboard only.

154. What is Monkey Testing?

> **Answer:** Random testing with unpredictable inputs and actions without any plan, to see whether the application crashes.
>
> **Example:** Tap random buttons very quickly on the product page, and see if the app hangs.

155. What is Alpha Testing?

> **Answer:** Testing by internal users or testers at the developer's site, before the release to external users, in a controlled setting.
>
> **Example:** Employees use the new checkout in the company's test environment.

156. What is Beta Testing?

> **Answer:** Testing by a limited group of real users in a real environment before the full release, to get feedback.
>
> **Example:** A group of 500 customers uses the new app version before the full launch.

157. What is the difference between Static Testing and Dynamic Testing?

> **Answer:** Static testing finds defects without running the code (reviews, walkthroughs, static analysis). Dynamic testing runs the code and checks the behaviour.
>
> **Example:** Static: reviewing the checkout requirement. Dynamic: executing a checkout with a real order.

Testing Fundamentals, SDLC & STLC (7)

158. What is the Software Development Life Cycle (SDLC)?

> **Answer:** The structured process for building software: Requirement, Design, Development, Testing, Deployment and Maintenance. Models include Waterfall, V-Model and Agile.
>
> **Example:** An e-commerce site goes from requirements > design > coding > testing > release > support.

159. What happens in the Requirement Analysis phase of STLC?

> **Answer:** Study the requirements to find testable points, clarify doubts with the BA/PO, identify test types, and prepare the RTM and the automation feasibility.
>
> **Example:** Read the coupon story, and ask 'is the discount applied before or after tax?'

160. What is covered in Test Planning and Strategy?

> **Answer:** Defining scope, approach, tools, environment, schedule, resources, risks, entry/exit criteria and effort estimates.
>
> **Example:** Plan 3 testers for 2 weeks for the new checkout, with Selenium for regression.

161. What happens in the Test Case Development phase?

> **Answer:** Writing test scenarios and detailed test cases with data and expected results, reviewing them, getting approval and updating the RTM.
>
> **Example:** Write 20 cases for 'Apply coupon' with positive, negative and boundary data.

162. How do you prepare Test Data?

> **Answer:** Create or mask data for the scenarios: users, products, coupons, payment cards. Use scripts, DB queries or API setup, cover valid, invalid and boundary data, and keep it refreshable.
>
> **Example:** Prepare a user with a Rs.0 wallet, an expired coupon and a product with only one unit in stock.

163. What happens in the Test Execution phase?

> **Answer:** Executing the test cases on the build, logging results, raising defects, retesting fixes, running regression and reporting the status daily.
>
> **Example:** Execute the checkout cases, log 3 defects and retest after the fixes.

164. What are the Test Closure activities?

> **Answer:** Final activities: confirm the exit criteria, review the open defects, prepare the test summary report, archive test artefacts and share lessons learned.
>
> **Example:** Release 5.2 report issued, the test cases archived and a retrospective held.

Scenario-Based Feature Testing (E-commerce & Real World) (17)

165. How do you test multi-currency payment in an e-commerce site?

> **Answer:** Verify the currency selection and conversion rates, rounding, display symbols, the charged currency vs the displayed one, the refund in the same currency, and fees. Validate with the gateway's sandbox.
>
> **Example:** A Rs.1,000 product shown as USD at the day's rate. The amount charged should match the checkout page.

166. How do you test order cancellation flow? **[asked x2 in your bank]**

> **Answer:** Test cancellation before and after shipping, full and partial cancel, status updates, the stock release, refund trigger and timeline, notifications, and cancellation of paid and COD orders.
>
> **Example:** Cancel an order before shipment: status becomes Cancelled, stock is restored and the refund starts.

167. How do you test order tracking system?

> **Answer:** Verify the status flow (Placed > Packed > Shipped > Out for delivery > Delivered), correct timestamps and tracking ID, the carrier link, notifications and the invalid IDs.
>
> **Example:** Tracking shows 'Shipped' only after the warehouse marks it so, and a wrong tracking ID shows an error.

168. How do you test email notifications after order confirmation? **[asked x2 in your bank]**

> **Answer:** Check it is sent for the right events, to the right recipient, with the correct content (order ID, items, amount, address), format on devices, links, timing, no duplicates, and spam folder issues.
>
> **Example:** After the payment succeeds, one email arrives within a minute, with the correct total and order ID.

169. How do you test price updates when a product goes on sale?

> **Answer:** Verify that the sale price shows everywhere (listing, product page, cart, checkout), starts and ends at the right time, the old price is struck through, and orders placed earlier keep their price.
>
> **Example:** At 12:00 AM the price changes from Rs.2,000 to Rs.1,500 on all pages, and after the sale ends it goes back.

170. How do you test stock management in an inventory system?

> **Answer:** Verify that the stock decreases after an order, increases after cancel/return, 'out of stock' shows at zero, low stock warnings, no negative stock, concurrent purchases and the sync between the warehouse and the site.
>
> **Example:** Stock 5, buy 2: stock becomes 3. Cancel the order and it returns to 5.

171. How do you test multi-user login sessions (same account on different devices)?

> **Answer:** Test logging in on two devices, whether it is allowed or the older session is ended, the session data consistency, the cart sync, logout from one device and the security alerts.
>
> **Example:** Log in on a phone and a laptop. The cart must be the same, and logging out on one should follow the defined rule.

172. How do you test login with social media (Google, Facebook)?

> **Answer:** Test successful login, denied permissions, cancelled login, a new vs existing account, email matching and merging, token expiry, logout and unlinking, and behaviour if the provider is down.
>
> **Example:** Click 'Login with Google', cancel on the consent screen, and the user returns to the login page with a message.

173. How do you test session timeout?

> **Answer:** Verify the session expires after the configured idle time, the user is redirected to login, the cart/data are handled as expected, the back button gives no access, and activity extends the session.
>
> **Example:** After 15 minutes idle, clicking Pay redirects to login without charging.

174. How do you test a shopping cart when multiple tabs are open?

> **Answer:** Test adding and removing items in different tabs, the sync of cart count, stale prices/stock, checkout in one tab and changes in another, and the final total.
>
> **Example:** Add an item in tab 1, and refresh tab 2. It should show the updated cart.

175. How do you test search suggestions (auto-complete)?

> **Answer:** Verify suggestions appear after the minimum characters, are relevant and ordered, handle typos, special characters, speed, keyboard and click selection, and no suggestions for no matches.
>
> **Example:** Typing 'sho' shows 'shoes', 'shorts'. Selecting 'shoes' opens the right results.

176. How do you test "Add to Wishlist" functionality? **[asked x2 in your bank]**

> **Answer:** Verify adding/removing items, login requirement, persistence, duplicates, moving to the cart, price/stock changes, a limit, and sharing.
>
> **Example:** Add a shirt to the wishlist, log out and log in again. It should still be there.

177. How do you test file upload functionality (e.g. profile picture upload)?

> **Answer:** Test allowed and disallowed types, size limit and boundaries, a corrupt file, an empty file, a long file name, special characters, multiple uploads, progress, cancel, and security (malicious files).
>
> **Example:** Upload a 2 MB JPG (allowed), a 6 MB image where the limit is 5 MB (rejected) and an .exe renamed to .jpg (rejected).

178. How do you test localization and internationalization of an e-commerce site (multi-language)? **[asked x3 in your bank]**

> **Answer:** Check translations, text expansion and layout, currency, date/number formats, right-to-left languages, character encoding, language switching and persistence, and local payment methods.
>
> **Example:** Switch to Hindi: labels, buttons and emails appear in Hindi, and Rs. and date formats are correct.

179. How do you test caching in e-commerce websites?

> **Answer:** Verify that the cached content updates when data changes (price, stock), the cache is cleared on logout/update, the stale content isn't shown, the headers/TTL, the browser vs the CDN cache, and the performance gains.
>
> **Example:** After a price change, the product page shows the new price without a manual hard refresh.

180. How do you test a new feature rollout using A/B testing?

> **Answer:** Verify that users are split correctly into variants (A/B), each sees a consistent experience, tracking/analytics events fire, there is no cross-over, and a rollback works.
>
> **Example:** 50% see the green Buy button, 50% see the blue. Clicks and orders are recorded per variant.

181. What would you test in an elevator?

> **Answer:** A fun scenario question: use a structured approach. Functional (floor buttons, doors, direction), non-functional (speed, safety, capacity), usability (accessibility, display), negative (overload, power cut, emergency button).
>
> **Example:** Press floor 5 and the lift stops at 5. Overload with extra weight and the alarm sounds, and the door stays open.

Integration, DB, API, Performance & Security Scenarios (5)

182. How do you test database rollback if a transaction fails?

> **Answer:** Force a failure in the middle of a transaction (for example, kill the connection after the payment is recorded but before the order is created) and verify that all changes are rolled back and the data is consistent.
>
> **Example:** If the order insert fails after the stock deduction, the stock must be restored.

183. How do you test data migration when moving from an old DB to a new DB?

> **Answer:** Compare the record counts, field-by-field data, relationships, null/duplicates, data types, performance and rollback. Use SQL checks, sampling and reconciliation reports before and after migration.
>
> **Example:** Check that the 1,00,000 customers and orders in the old DB match the new DB, including addresses and totals.

184. How do you test performance of the product search API?

> **Answer:** Define the response time target and test the API with the JMeter/Postman under different loads and queries, check the response time, throughput, errors, DB query time and caching.
>
> **Example:** The search API should respond in under 500 ms with 1,000 requests per second.

185. How do you test performance of bulk order processing?

> **Answer:** Test processing time and the throughput with large order volumes, queues, DB locks, memory/CPU, failure handling, retries, partial failures and the data accuracy.
>
> **Example:** Process 10,000 orders in a batch, and check that all are created once, with no duplicates or losses.

186. What if the API response is slow but the UI has no timeout handling?

> **Answer:** That is a defect in resilience. Simulate a slow response (with a proxy/mock), observe the UI (loader, timeout, error message), and report the missing timeout handling, so the app shows a message and a retry.
>
> **Example:** A 30-second delay on the payment API leaves the user on a blank screen. Expected: a loader, then a timeout message with a retry option.

Metrics, Coverage & Reporting (4)

187. What is Test Case Effectiveness and how do you measure it? **[asked x2 in your bank]**

> **Answer:** How well the test cases find defects: (defects found by test cases / total defects) x 100. Higher is better, and it shows the quality of the test design.
>
> **Example:** Test cases found 45 of 50 total defects, so effectiveness is 90%.

188. What is Defect Removal Efficiency (DRE)?

> **Answer:** The percentage of defects removed before release: defects found before release / (before + after release) x 100.
>
> **Example:** 95 defects before release and 5 in production gives 95% DRE.

189. What is Requirement Coverage?

> **Answer:** The percentage of requirements covered by test cases: (requirements covered / total requirements) x 100, tracked with the RTM.
>
> **Example:** Out of 40 requirements, 38 have test cases, so coverage is 95%.

190. What is Code Coverage?

> **Answer:** The percentage of code (statements, branches, paths) executed by tests, measured by tools such as JaCoCo. It shows the untested code, but not the quality of the tests.
>
> **Example:** Unit tests execute 80% of the discount module's code.

JIRA & Defect Tracking Tools (6)

191. What is a Workflow in JIRA?

> **Answer:** The set of statuses and transitions an issue passes through, from creation to completion, in JIRA.
>
> **Example:** Bug workflow: Open > In Progress > Ready for QA > Closed (or Reopened).

192. What is an Issue in JIRA?

> **Answer:** A single work item in JIRA, such as a Bug, Story, Task, Epic or Sub-task.
>
> **Example:** A 'Bug' issue: WISH-102 'Wishlist count wrong'.

193. Can JIRA be used for Test Case Management?

> **Answer:** Yes, with plug-ins such as Xray or Zephyr (which add test cases, executions and reports), or through custom issue types. Plain JIRA is not a full test management tool.
>
> **Example:** With Xray, create the test cases for 'Coupon', link them to the story and track the execution.

194. How do you link issues in JIRA?

> **Answer:** Open the issue, click 'Link', pick a link type (blocks, is blocked by, relates to, duplicates) and choose the target issue.
>
> **Example:** A bug 'blocks' the story and is linked to the test case that found it.

195. What are JIRA Filters and how can QA use them?

> **Answer:** Saved searches written in JQL to list issues by conditions. QA uses them for open bugs, my bugs, bugs by sprint, and dashboards.
>
> **Example:** Filter: project = SHOP AND type = Bug AND status != Closed AND sprint in openSprints().

196. How do you generate reports in JIRA?

> **Answer:** Use built-in reports (burndown, velocity, created vs resolved), dashboards with gadgets, filters/exports to Excel, and test plugin reports.
>
> **Example:** A dashboard showing open bugs by severity for the release.

QA Mindset, Leadership & Career (8)

197. What does quality mean to you as a QA engineer?

> **Answer:** Quality means the product meets user needs and is reliable, secure and usable, not just 'zero bugs'. I see QA as helping the whole team build quality in.
>
> **Example:** A fast, clear and safe checkout that makes the customer trust the site.

198. How do you ensure quality beyond automation?

> **Answer:** By taking part in requirement reviews, exploratory and usability testing, risk analysis, monitoring production, working with developers on testability and promoting good practices.
>
> **Example:** I review stories early, and watch the production error logs after the release.

199. How do you balance manual and automation tasks?

> **Answer:** Automate stable, repetitive, high-value cases and use manual effort for new features, exploratory and usability testing, based on risk and time.
>
> **Example:** Automated regression runs nightly, while I explore the new gift card feature manually.

200. How do you build trust with developers?

> **Answer:** By being reliable, respectful and clear: precise bug reports, no blame, early collaboration, sharing test insights and acknowledging their fixes.
>
> **Example:** I demo the bug on a call and not just through a ticket, and thank the dev for the quick fix.

201. How do you contribute to a bug prevention mindset?

> **Answer:** By shifting left (requirement reviews), doing RCA of leaks, sharing the common defect patterns, adding checks to the DoD, and suggesting unit tests and code review checklists.
>
> **Example:** After a rounding bug, we add rounding cases to the DoD for every price story.

202. How do you mentor junior testers?

> **Answer:** By pairing, giving clear tasks with feedback, sharing checklists and the domain knowledge, reviewing their test cases, and encouraging them to ask questions.
>
> **Example:** I let a junior write the wishlist cases and review them with tips on boundary and negative cases.

203. What was your proudest QA contribution?

> **Answer:** Use a real example with a measurable outcome: for example, building a regression suite, reducing the leakage, or improving the process.
>
> **Example:** I automated the checkout regression, cutting the release test time from 2 days to 4 hours.

204. Where do you see your career in QA heading?

> **Answer:** Show growth with a plan: deepen the skills in automation, API and performance testing, take a lead role, mentor, and move toward a quality-owner or test-lead role.
>
> **Example:** Within 2 to 3 years, lead the QA for an e-commerce product and drive its automation strategy.

Section 4: Low Priority / Quick Revision (30 questions)

Fresher-level definitions and niche topics. Rarely asked at 4+ years, but worth a one-line answer so you are never caught off guard.

Defect Management & Bug Life Cycle (5)

205. What is a Bug/Defect in Software Testing?

> **Answer:** A flaw in the software where the actual behaviour differs from the expected behaviour.
> **Example:** The cart total does not include the shipping charge.

206. What is the purpose of logging defects early?

> **Answer:** Early detection makes fixes cheaper and faster, prevents the bug from spreading to other modules and avoids late surprises.
> **Example:** A wrong price rule found in the sprint costs far less than after the sale launch.

207. What is a Latent Defect?

> **Answer:** A defect that exists in the released software but is not yet found, and appears only under certain conditions later.
> **Example:** A bug that appears only when a coupon is used in a particular time zone.

208. What is a Masked Defect?

> **Answer:** A defect hidden by another defect, and found only after the first one is fixed.
> **Example:** A payment page crash hides the wrong tax calculation behind it.

209. What is a Bug Life Cycle Tool? Give examples.

> **Answer:** Tools used to log, track and manage defects through their life cycle. Examples: JIRA, Bugzilla, Azure DevOps, Mantis, Redmine.
> **Example:** I log the checkout bug in JIRA and track it to closure.

Smoke / Sanity / Regression & Automation Decisions (2)

210. What is Smoke Testing? Why is it called so?

> **Answer:** A quick check of the critical functions to confirm the build is stable enough for further testing. The name comes from hardware testing: switch on the device and see if smoke comes out.
> **Example:** After a new build: open the site, log in, search, and add to cart.

211. How would you cover character keyboard operation from the context menu using a user-defined keyword? (legacy keyword-driven tool question)

> **Answer:** In a keyword-driven framework, create a user-defined keyword (for example, 'PressKey') that takes the key and the target element and triggers the context menu action, then reuse it in the test scripts with different data.
> **Example:** A keyword 'RightClickAndSelect' used on the product image to select 'Open in new tab'.

Test Design, Planning & Documentation (1)

212. What is the purpose of Test Documentation?

> **Answer:** Provides traceability, a clear record of what was tested, supports reviews and audits, helps new team members and makes the reuse of tests easier.
> **Example:** Test plan, test cases and reports show how the checkout was verified.

Types & Levels of Testing (8)

213. What is Scalability Testing?

> **Answer:** Testing how well the system handles growth in users, data or transactions, by scaling up or out.
> **Example:** Does the site still perform when the daily orders grow from 10,000 to 100,000?

214. What is Stability Testing?

> **Answer:** Testing the system's ability to run reliably over a long period under a normal load without failures or degradation.
> **Example:** Run the site with normal traffic for 48 hours and check for memory leaks.

215. What is Installation Testing?

> **Answer:** Verifying that the software installs, upgrades and uninstalls correctly on the supported environments.
> **Example:** Install the shopping app on Android and iOS, then update and uninstall it.

216. What is Recovery Testing?

> **Answer:** Testing how well the system recovers from crashes, hardware failure or network loss, without data loss.
> **Example:** Kill the server during an order, restart, and check that the order data is intact.

217. What is Compliance Testing?

> **Answer:** Verifying that the product follows the required standards, laws and regulations.
> **Example:** Payment pages follow PCI-DSS, and the site meets data privacy rules.

218. What is Internationalization Testing?

> **Answer:** Checking that the product can be adapted to different languages and regions without code changes (design and readiness for localisation).
> **Example:** The UI supports text expansion, Unicode, and different date and currency formats.

219. What is Configuration Testing?

> **Answer:** Testing the software with different configurations of hardware, software, OS and browsers to find the compatible ones.
> **Example:** Checkout on Windows/Chrome, macOS/Safari and Android 12/13.

220. What is Pair Testing?

> **Answer:** Two people (for example, a tester and a developer or two testers) work together on one machine to test, sharing ideas and finding defects faster.
> **Example:** A tester and a developer explore the new checkout together.

Testing Fundamentals, SDLC & STLC (1)

221. What is Software Testing?

> **Answer:** The process of evaluating software to find defects and to verify that it meets requirements and works as expected.
> **Example:** Placing test orders to check that the site works properly before the release.

Scenario-Based Feature Testing (E-commerce & Real World) (2)

222. How will you test cart abandonment recovery in an e-commerce site?

> **Answer:** Verify that the abandoned cart is saved, the reminder email/notification is sent at the right time with the correct items and prices, the link returns the user to the cart, the coupon (if any) works, and no reminder after the purchase.
> **Example:** Leave the items in the cart for 24 hours, and a reminder email arrives with a working 'Return to cart' link.

223. How do you validate image rendering via UI tests?

> **Answer:** Check that images load (no broken links), correct image per product, the size and quality, alt text, responsive scaling, lazy loading, and the visual comparison with tools such as Applitools or screenshot compare.
>
> **Example:** On the product page, the correct main image and thumbnails appear on desktop and mobile.

Integration, DB, API, Performance & Security Scenarios (1)

224. How do you test log generation in applications?

> **Answer:** Verify that the logs are created for key events and errors, with the correct level, timestamp, user/order ID, no sensitive data (passwords, card numbers), rotation and the proper format.
>
> **Example:** A failed payment writes an ERROR log with the order ID but no card number.

Metrics, Coverage & Reporting (5)

225. What is Test Execution Status?

> **Answer:** A summary of how many test cases are passed, failed, blocked, not run or in progress at a given time.
>
> **Example:** Today: 200 passed, 20 failed, 10 blocked, 70 not run.

226. What is Test Productivity?

> **Answer:** The amount of testing done in a time period, for example, test cases executed per day or per person.
>
> **Example:** A tester executes 25 test cases per day.

227. What is Mean Time to Detect (MTTD)?

> **Answer:** The average time taken to detect a problem after it was introduced or occurred. Lower is better.
>
> **Example:** A pricing bug introduced on Monday is found on Tuesday, so the MTTD is 1 day.

228. What is Mean Time to Repair (MTTR)?

> **Answer:** The average time taken to fix a problem after it is detected, until it is resolved.
>
> **Example:** Production payment failures are fixed in 3 hours on average.

229. How do you maintain consistency in test reports?

> **Answer:** Use a standard template, common definitions and metrics, consistent naming and statuses, a single source (JIRA/test tool), and review before sharing.
>
> **Example:** Every sprint report uses the same layout, severity scale and pass rate formula.

QA Mindset, Leadership & Career (5)

230. What is your approach to giving feedback in peer reviews?

> **Answer:** Be specific, kind and constructive: start with what is good, point out the issue with an example and a suggestion, and focus on the work, not the person.
>
> **Example:** 'The login cases are clear. Adding a lockout negative case would improve coverage.'

231. How do you help developers write testable code?

> **Answer:** By reviewing the design early, asking for test hooks (IDs, logs, APIs), small modular functions, clear error handling, good unit tests and avoiding hard-coded data.
>
> **Example:** Ask for unique IDs on checkout fields so that automation is stable.

232. What is your role in the sprint retrospective as a QA?

> **Answer:** I share the testing challenges, such as late builds or unclear requirements, give data (defect trends), propose improvements and take ownership of the action items.
>
> **Example:** Suggest to demo each story to QA before it moves to testing.

233. How do you upskill your QA team?

> **Answer:** Through knowledge-sharing sessions, training and certifications, pair testing, documented best practices, a learning plan for each person, and practising on real defects.
>
> **Example:** A monthly session on API testing using the real checkout APIs.

234. What books, blogs, or courses have shaped your QA thinking?

> **Answer:** Name genuine resources you have used, and the lesson you took from them. Examples: ISTQB syllabus, Lessons Learned in Software Testing, Ministry of Testing and Test Automation University, with an example of how you applied them.
>
> **Example:** A course on API testing made me add Postman checks for the order service, which found price mismatches earlier.